# Insertion Sort

**Resumo completo** — baseado em `Aula6_AnaliseOrdenacao.pdf` e `Lista6_AnaliseOrdenacao.pdf` (Exercício 5; partes dos Exercícios 1, 7, 8 e 14).

| Propriedade | Valor |
|---|---|
| Estratégia | constrói uma região ordenada e **insere** cada novo elemento na posição correta |
| Melhor caso | $\Theta(n)$ — vetor já ordenado |
| Caso médio | $\Theta(n^2)$ |
| Pior caso | $\Theta(n^2)$ — ordem decrescente |
| Espaço auxiliar | $\Theta(1)$ |
| In-place | Sim |
| Estável | Sim (desloca só com `>`) |

---

## 1. Ideia

É como organizar **cartas na mão**: pega a próxima carta e a coloca no lugar certo entre as que já estão ordenadas, **deslocando** as maiores uma posição para a direita para abrir espaço.

### Passo a passo em `[5, 2, 4, 1]`

| Inserindo | Ação | Vetor (região ordenada em negrito) |
|---|---|---|
| 5 | primeiro elemento: já é uma região ordenada | [**5**, 2, 4, 1] |
| 2 | desloca 5 → insere 2 na pos 0 | [**2, 5**, 4, 1] |
| 4 | desloca 5 → insere 4 na pos 1 | [**2, 4, 5**, 1] |
| 1 | desloca 5, 4, 2 → insere 1 na pos 0 | [**1, 2, 4, 5**] |

Total: 1 + 1 + 3 = 5 deslocamentos (= 5 inversões do vetor).

---

## 2. Implementação

```
def insertion_sort(v):
    for i in range(1, len(v)):
        chave = v[i]                        # elemento a inserir
        j = i - 1
        while j >= 0 and v[j] > chave:      # verificação de limite + comparação
            v[j + 1] = v[j]                 # desloca o maior para a direita
            j -= 1
        v[j + 1] = chave                    # insere na posição correta
    return v
```

- **Invariante:** no início da iteração `i`, `v[0..i-1]` está ordenado.
- **Estabilidade:** a condição é `v[j] > chave` (e não `>=`). Um elemento igual à `chave` **não** é deslocado, então a chave fica depois dele — a ordem relativa de iguais é preservada.

---

## 3. Análise de complexidade passo a passo

Seja $t_i$ o número de elementos de `v[0..i-1]` **maiores** que `v[i]` (quantos serão deslocados na iteração $i$).

- Deslocamentos na iteração $i$: $t_i$.
- Comparações `v[j] > chave` na iteração $i$: $t_i + 1$ (a última comparação é a que falha), ou só $t_i$ se a chave for até o início (`j` chega a $-1$).

Cada deslocamento corrige exatamente uma inversão, logo:

$$\text{deslocamentos} = \sum_{i=1}^{n-1} t_i = I \quad (\text{nº de inversões}), \qquad T(n) = \Theta(n + I)$$

### 3.1 Melhor caso — vetor já ordenado

$t_i = 0$ para todo $i$: cada iteração faz **uma** comparação (que falha) e nenhum deslocamento.

$$T(n) = \sum_{i=1}^{n-1} c = c(n-1) = \Theta(n)$$

### 3.2 Pior caso — ordem decrescente

Cada novo elemento é menor que todos os anteriores: $t_i = i$.

$$\sum_{i=1}^{n-1} t_i = 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2} \quad \Rightarrow \quad T(n) = \Theta(n^2)$$

### 3.3 Caso médio — permutação aleatória

Em média, metade dos $i$ elementos anteriores é maior que `v[i]`: $E[t_i] = i/2$.

$$\sum_{i=1}^{n-1} \frac{i}{2} = \frac{n(n-1)}{4} \quad \Rightarrow \quad T(n) = \Theta(n^2)$$

(Metade do pior caso, mas a mesma ordem de crescimento.)

### 3.4 Espaço

Só `chave`, `i` e `j`: $\Theta(1)$ → **in-place**.

### 3.5 Por que é bom para vetores pequenos ou quase ordenados?

- Custo $\Theta(n + I)$: se o vetor está **quase ordenado**, há poucas inversões ($I$ pequeno) e o custo fica perto de linear.
- Código simples, sem recursão, constantes pequenas: para $n$ pequeno, ganha de algoritmos $n \log n$ (bibliotecas reais usam Insertion Sort para pedaços pequenos dentro de Merge/Quick Sort).
- Estável, in-place e pode ordenar à medida que os dados chegam (cada novo elemento é só mais uma inserção).

---

## 4. Exercícios resolvidos

### Exercício 5 — `v = [31, 12, 45, 7, 26, 18, 39, 4, 22, 15]` ($n = 10$)

**a/b) Vetor após cada iteração do laço externo e elementos deslocados:**

| $i$ | Chave | Deslocados | Nº | Vetor após a iteração |
|---|---|---|---|---|
| 1 | 12 | 31 | 1 | `[12, 31, 45, 7, 26, 18, 39, 4, 22, 15]` |
| 2 | 45 | — | 0 | `[12, 31, 45, 7, 26, 18, 39, 4, 22, 15]` |
| 3 | 7 | 45, 31, 12 | 3 | `[7, 12, 31, 45, 26, 18, 39, 4, 22, 15]` |
| 4 | 26 | 45, 31 | 2 | `[7, 12, 26, 31, 45, 18, 39, 4, 22, 15]` |
| 5 | 18 | 45, 31, 26 | 3 | `[7, 12, 18, 26, 31, 45, 39, 4, 22, 15]` |
| 6 | 39 | 45 | 1 | `[7, 12, 18, 26, 31, 39, 45, 4, 22, 15]` |
| 7 | 4 | 45, 39, 31, 26, 18, 12, 7 | 7 | `[4, 7, 12, 18, 26, 31, 39, 45, 22, 15]` |
| 8 | 22 | 45, 39, 31, 26 | 4 | `[4, 7, 12, 18, 22, 26, 31, 39, 45, 15]` |
| 9 | 15 | 45, 39, 31, 26, 22, 18 | 6 | `[4, 7, 12, 15, 18, 22, 26, 31, 39, 45]` |

Total de deslocamentos: $1 + 0 + 3 + 2 + 3 + 1 + 7 + 4 + 6 = 27$ (= inversões do vetor).

**c) Inserção do 4: 7 deslocamentos** (todos os 7 elementos anteriores são maiores; o 4 vai para a posição 0).

**d) Comparações.** Contando separadamente as duas partes da condição `j >= 0 and v[j] > chave`:

| $i$ | Chave | Comparações entre elementos (`v[j] > chave`) | Verificações de limite (`j >= 0`) | Observação |
|---|---|---|---|---|
| 1 | 12 | 1 | 2 | chegou ao início (`j = -1`) |
| 2 | 45 | 1 | 1 | |
| 3 | 7 | 3 | 4 | chegou ao início |
| 4 | 26 | 3 | 3 | |
| 5 | 18 | 4 | 4 | |
| 6 | 39 | 2 | 2 | |
| 7 | 4 | 7 | 8 | chegou ao início |
| 8 | 22 | 5 | 5 | |
| 9 | 15 | 7 | 7 | |
| **Total** | | **33** | **36** | |

Conferência: comparações entre elementos = deslocamentos + inserções que pararam por encontrar um menor = $27 + 6 = 33$. Verificações de limite = $33 + 3$ (as 3 vezes em que `j >= 0` foi falso e o `and` nem chegou a comparar elementos) = 36.

**e)** Melhor caso $\Theta(n)$ (vetor ordenado: $n - 1$ comparações); pior caso $\Theta(n^2)$ (ordem decrescente: $n(n-1)/2$ comparações e deslocamentos).

**f)** Para vetores pequenos ou quase ordenados, o número de inversões é pequeno e o custo $\Theta(n + I)$ fica próximo de linear, com código simples, sem memória extra e estável (seção 3.5).

### Exercício 1 (parte Insertion) — `[7, 3, 5, 2]`

1ª iteração (`chave = 3`): 7 > 3 → desloca 7; insere 3 na pos 0 → `[3, 7, 5, 2]` (1 comparação). Resolução completa em 02 — Fundamentos.

### Exercício 7 (parte Insertion)

| Vetor | Comparações entre elementos | Deslocamentos |
|---|---|---|
| `v1 = [16, 9, 28, 3, 21, 7, 35, 12, 5, 24]` | 29 | 22 |
| `v2` (já ordenado) | **9** ($= n - 1$) | 0 |

### Exercícios 8 e 14 — onde o Insertion aparece

- **Ex. 8:** é um dos algoritmos estáveis que pode ordenar os funcionários por salário preservando a ordem entre iguais.
- **Ex. 14, cenário A** (10 elementos, quase ordenado): é a escolha recomendada.

Resoluções completas em 02 — Fundamentos.

---

## Matérias correlatas

**Somas** $\sum i = n(n-1)/2$ e $\sum i/2 = n(n-1)/4$: apêndice de 01 — Recursividade.

**Inversões:** deslocamentos do Insertion = trocas do Bubble = inversões. Veja 02 — Fundamentos e 03 — Bubble Sort.

**Merge Sort** também é estável, mas $\Theta(n \log n)$ no pior caso, à custa de $\Theta(n)$ de memória. Veja 06 — Merge Sort.